

Arduino Track – Deep Dive 2026

BASUONI TEAM

Team members :

- Abdullah Mohammad Muslim
- Farah Sabri Mohammad
- Sara Ahmed Elmesalamy
- Shahd Mamdouh Mahmoud
- Youssef Ali Abdullah

Table of Contents

1. Project Overview

1.1 Project Description

1.2 Problem Statement

1.3 Project Objectives

1.4 Main Features

2. System Architecture

2.1 Overall System Block Diagram

2.2 Explanation of Main System Components

2.3 Communication Between Components

3. Hardware Design

3.1 Components List

3.2 Circuit Diagram

3.3 Wiring

3.4 Sensor Configuration

3.5 Motor and Motor Driver Configuration

3.6 Power System

4. Design Calculations

5. Software Design

5.1 Software Architecture

5.2 Main Program Logic

5.3 Manual Mode Implementation

5.4 Autonomous Mode Implementation

5.5 Control Algorithm

1. Project Overview

1.1 Project Description

The project is a mobile robotic system designed to operate in two different modes: Manual Mode and Autonomous Line-Following Mode. The robot is based on an ESP32 microcontroller and uses five IR sensors to detect and track the designated line during Autonomous Mode. The robot is driven by two DC motors, while a three-battery holder provides the required power for the system. The project integrates embedded programming, sensor interfacing, motor control, and system integration into one complete robotic system.

1.2 Problem Statement

The main challenge of the project is to develop a robot capable of completing the designated track accurately, consistently, and efficiently in both operating modes. In Autonomous Mode, the robot must detect the line and adjust its movement without human intervention. In Manual Mode, the robot must respond accurately to the wireless controller while maintaining stable and controlled movement.

1.3 Project Objectives

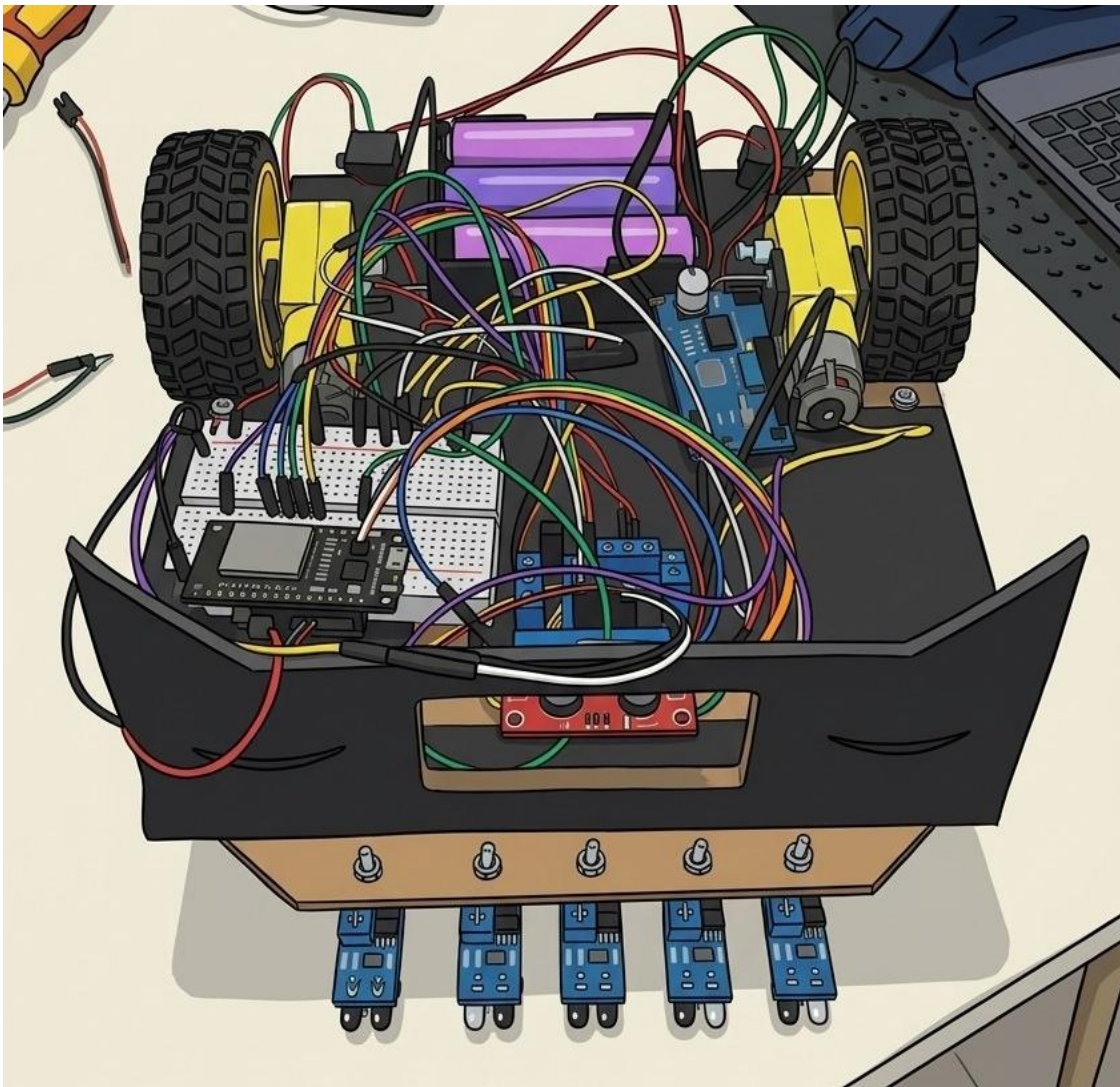
The main objectives of this project are:

- Design and build a complete mobile robot.
 - Implement both Manual and Autonomous operating modes.
 - Interface five IR sensors with the ESP32.
 - Control two DC motors efficiently.
 - Implement motor speed and direction control.
 - Develop an autonomous line-following algorithm.
 - Implement wireless manual control.
 - Test, debug, and improve the complete robotic system.
 - Achieve accurate, stable, and consistent movement on the designated track.
-

1.4 Main Features

The robot includes the following features:

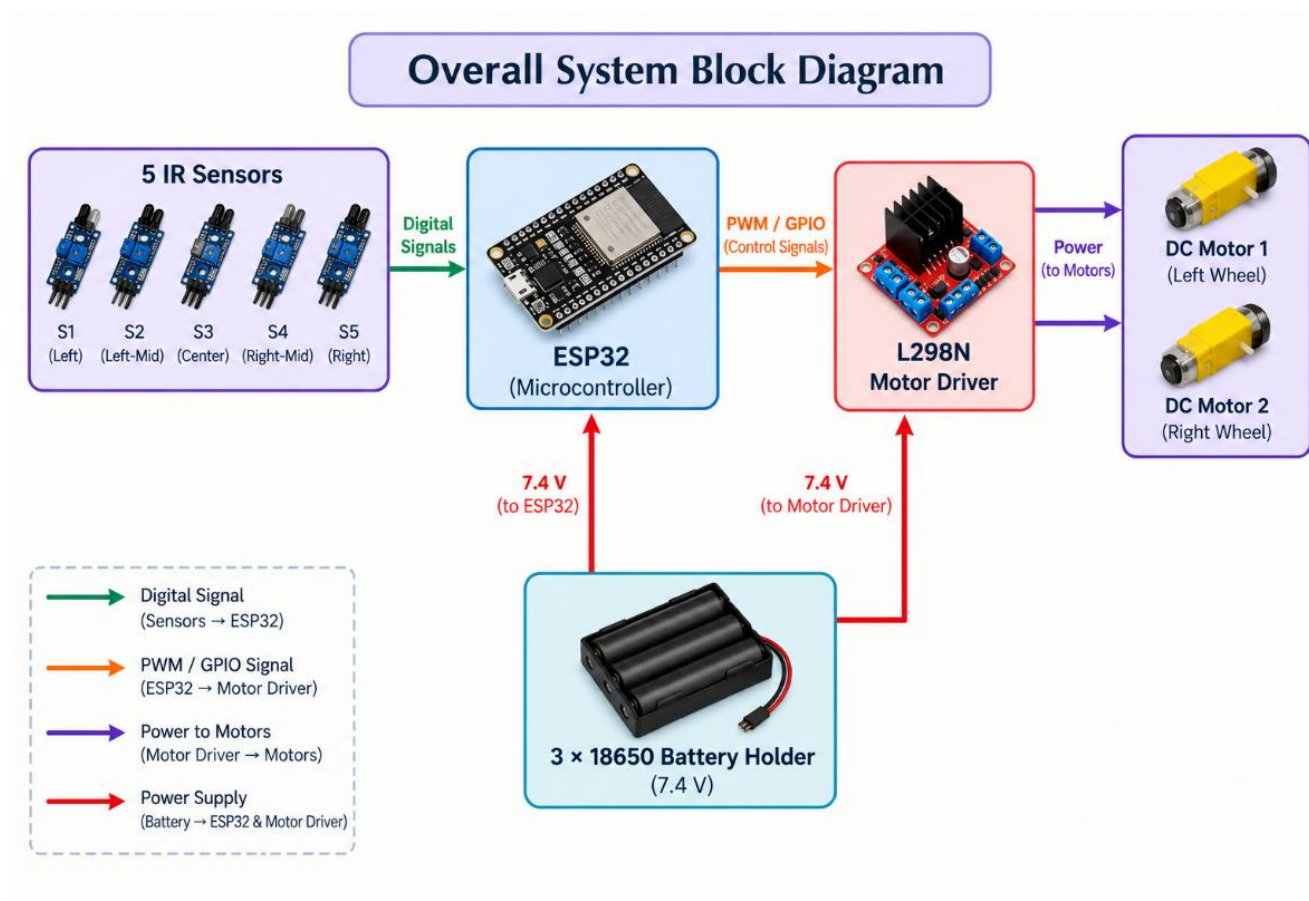
- Dual operating modes: Manual and Autonomous.
- Five IR sensors for line detection and tracking.
- ESP32-based control system.
- Two DC motors for robot movement.
- Wireless control in Manual Mode.
- Automatic line following in Autonomous Mode.
- Motor speed and direction control.
- Battery-powered operation using a three-battery holder.
-



2. System Architecture

2.1 Overall System Block Diagram

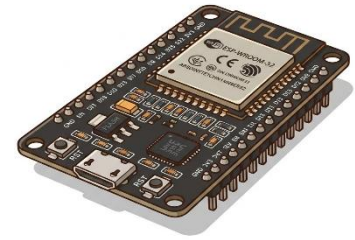
The robot is controlled by an ESP32 microcontroller. Five IR sensors provide line-position information to the ESP32, which processes the sensor readings and determines the required movement during Autonomous Mode. The ESP32 also receives commands from the wireless controller during Manual Mode. Based on the selected mode, the ESP32 sends control signals to the motor driver, which controls the two DC motors. The complete system is powered by a three-battery holder.



2.2 Explanation of Main System Components

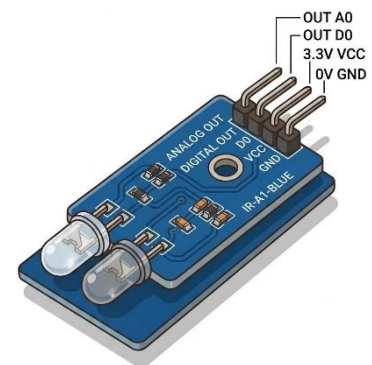
ESP32

The ESP32 acts as the main control unit of the robot. It processes the sensor readings, handles the operating modes, and generates the required control signals for the system.



IR Sensors

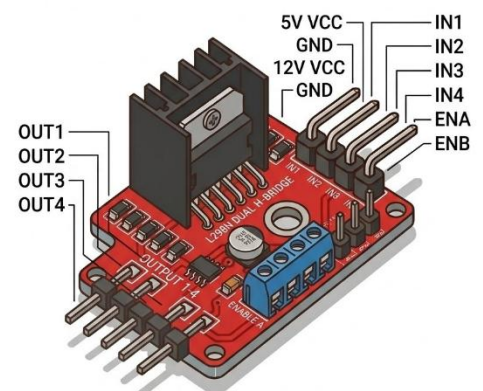
Five IR sensors are used to detect the position of the line. They are arranged across the front of the robot to provide information about the robot's alignment with the track.



Motor Driver

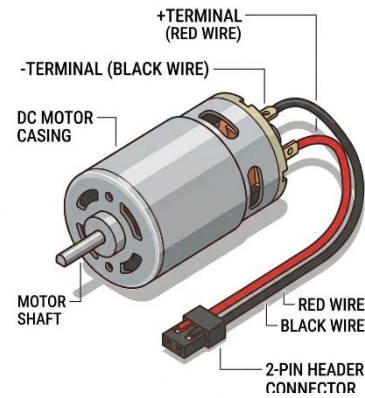
The motor driver provides the interface between the ESP32 and the two DC motors. It receives control signals from the ESP32 and controls the motors accordingly.

L298 MOTOR DRIVER MODULE



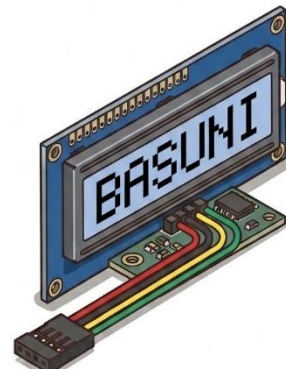
DC Motors

Two DC motors provide the mechanical movement of the robot. Their speed and direction are controlled to achieve forward movement and turning.



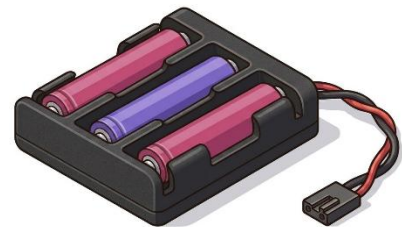
LCD Display

The LCD is used to display system information and provide visual feedback during robot operation.



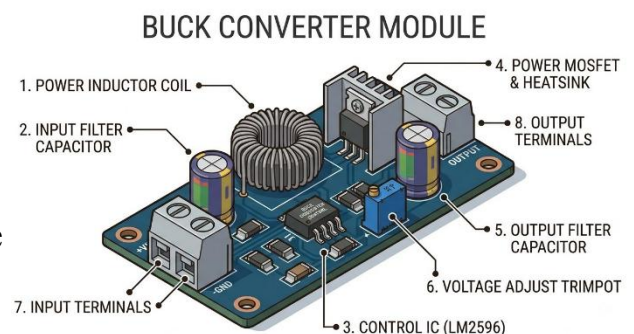
Battery Holder

A three-battery holder is used as the main onboard power source for the robot.



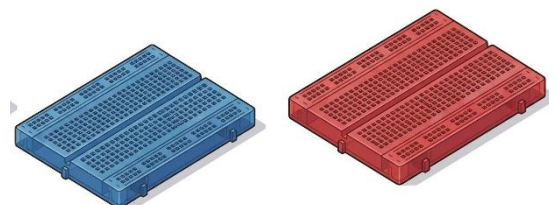
Buck Converter

The buck converter is used to regulate the input voltage from the battery supply to the appropriate voltage required by the electronic components.



Breadboard

The breadboard is used to facilitate temporary electrical connections between electronic components during assembly, testing, and debugging.



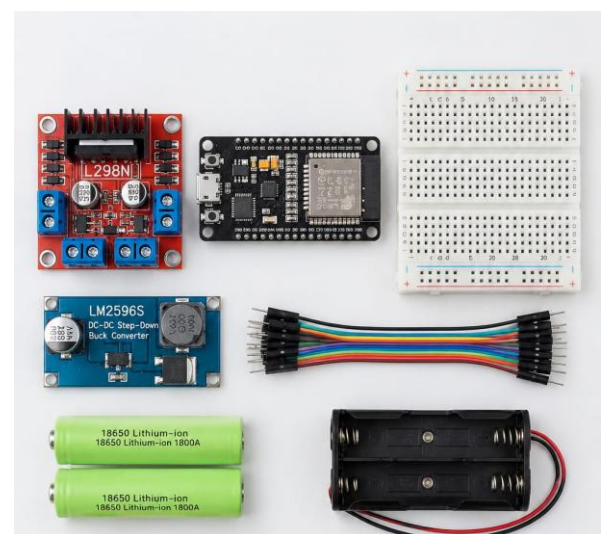
2.3 Communication Between Components

Data is exchanged between the system components using different communication methods, while the IR sensors send digital signals to the controller. After processing the data, the microcontroller generates control and PWM signals that are sent to the motor driver, which then controls the speed and direction of the motors.

3. Hardware Design

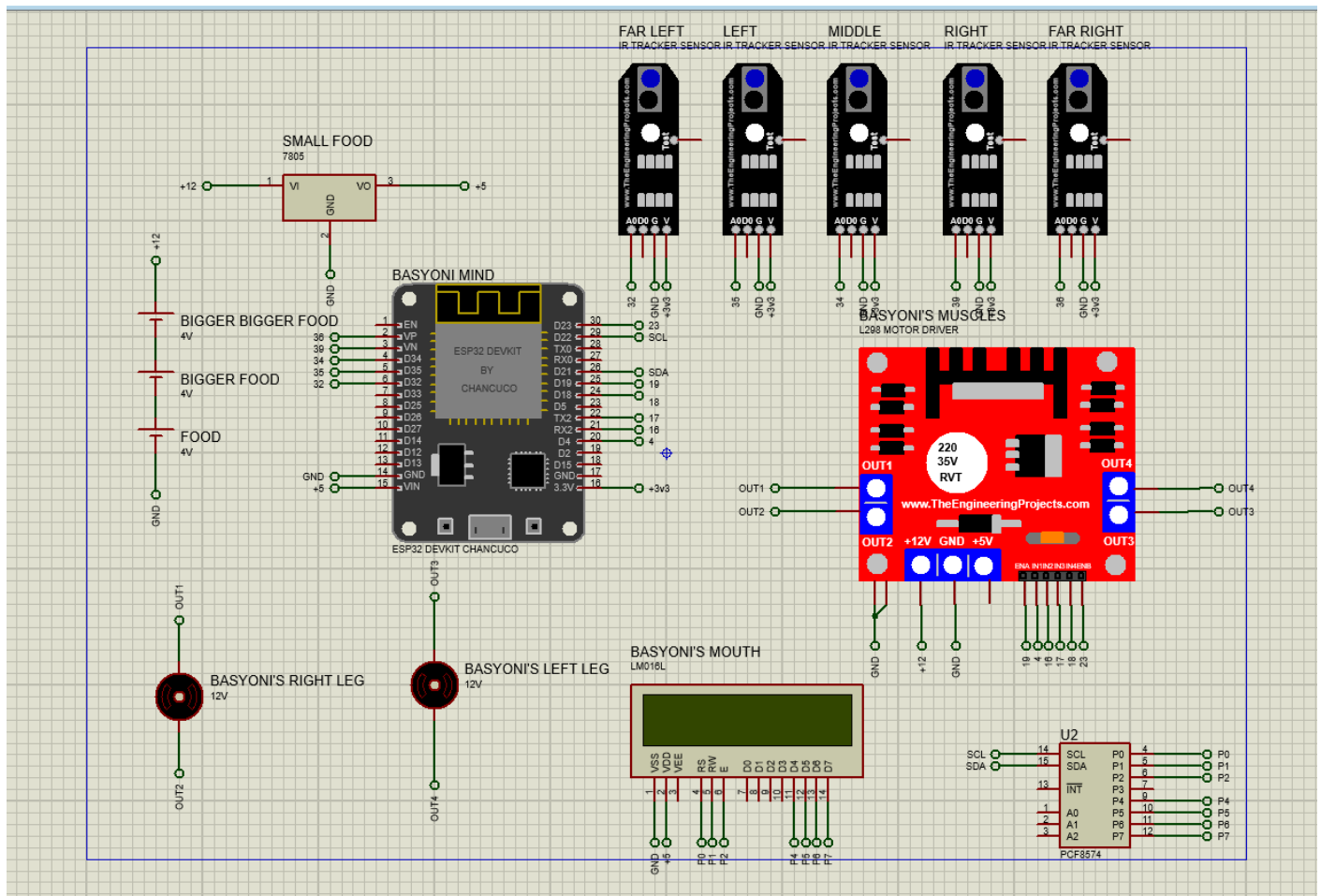
3.1 Components List

The robot consists of several electronic and mechanical components working together to achieve the required functionality. These components include a microcontroller, motor driver, Bluetooth module, three IR sensors, four DC motors, a battery pack, a chassis, wheels, and connecting wires.



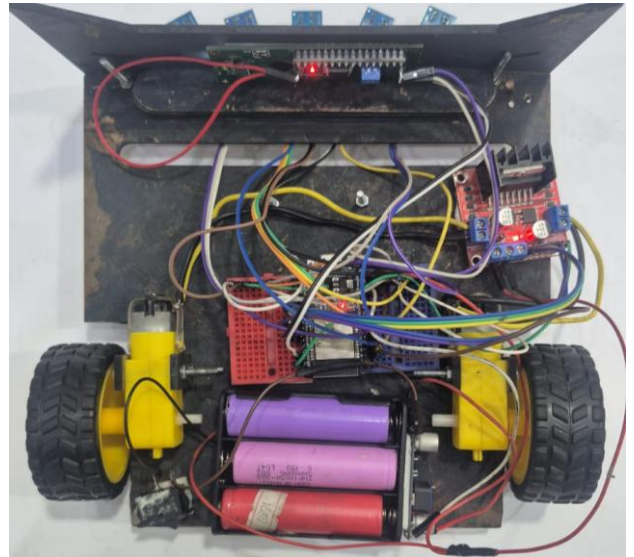
3.2 Circuit Diagram

The circuit diagram illustrates the electrical connections between all system components, including the sensors, Bluetooth module, motor driver, and power supply. It provides a clear understanding of how the different parts of the system are interconnected.



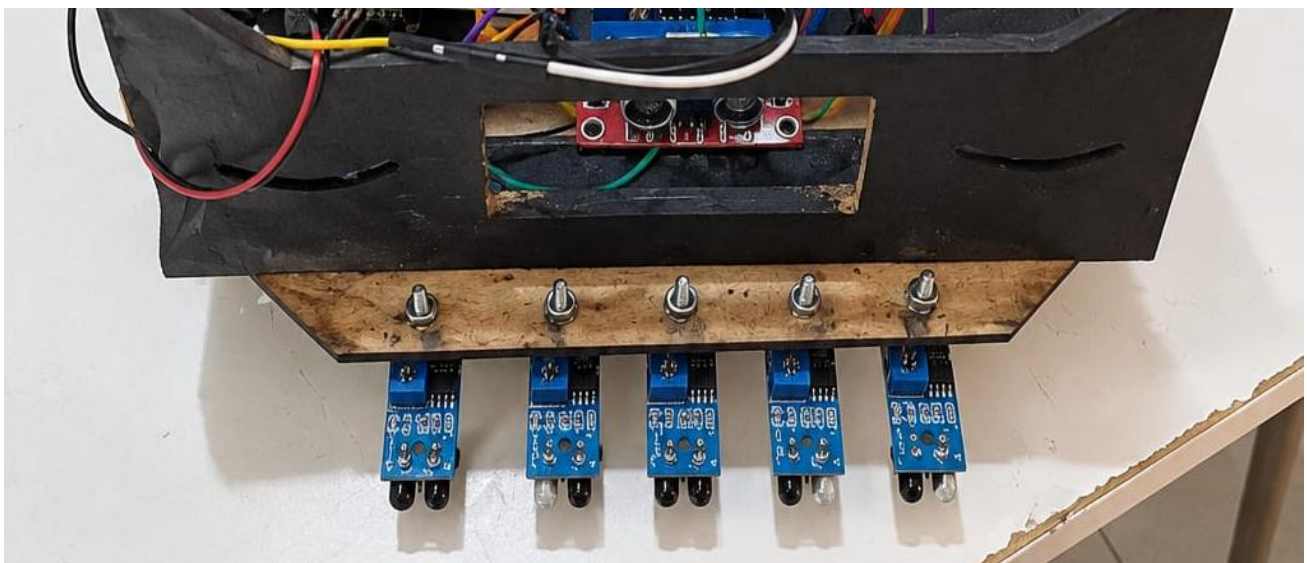
3.3 Wiring

All components are connected to the microcontroller according to the system design requirements. The connections include linking the sensors to input pins , connecting the Bluetooth module through serial communication pins, and interfacing the motor driver with the microcontroller output pins.



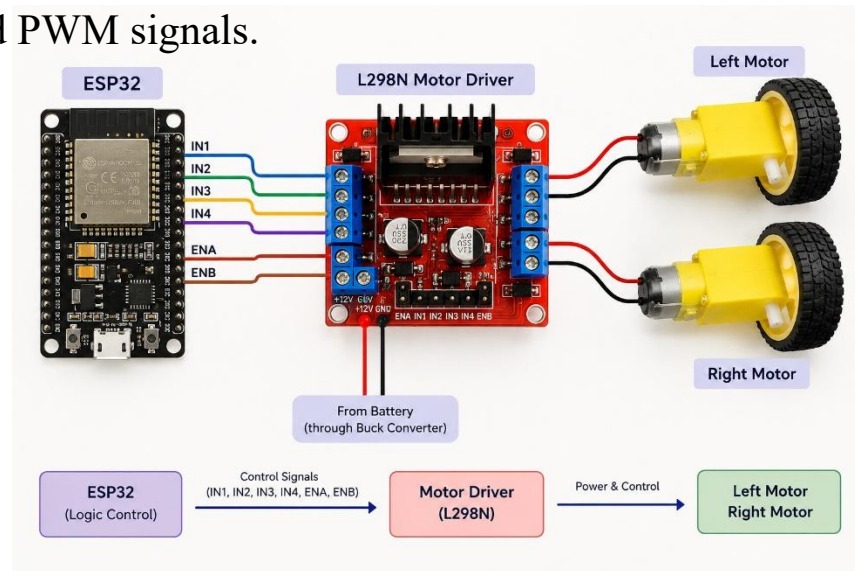
3.4 Sensor Configuration

Three IR sensors are mounted at the front of the robot. The center sensor detects the line directly beneath the robot, while the left and right sensors help determine any deviation from the path and allow the robot to correct its direction accordingly.



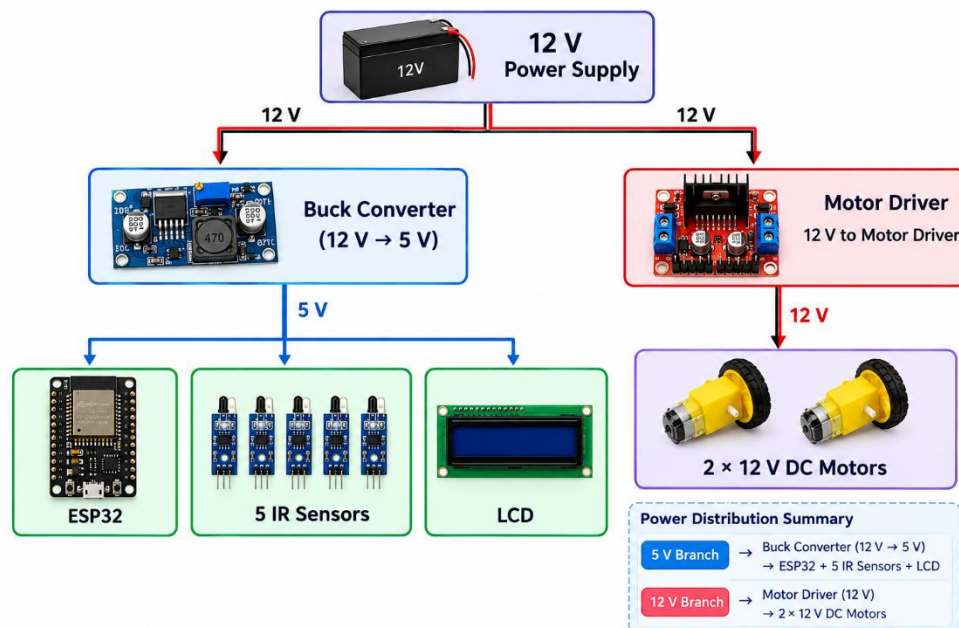
3.5 Motor and Motor Driver Configuration

The DC motors are controlled through a motor driver that receives control signals from the microcontroller. This configuration allows the system to control both motor speed and direction using digital control signals and PWM signals.



3.6 Power System

The robot uses a **12 V power supply** as the main power source. The power system is divided into two branches. The first branch is connected to a **buck converter**, which reduces the input voltage from **12 V to 5 V**. The 5 V output is used to supply the **ESP32, five IR sensors, and LCD**. The second branch is connected directly to the **motor driver**, which supplies the required **12 V** power to the **two DC motors**.



4. Design Calculations

4.1 Voltage Regulation

The main power supply provides:

$$V_{in} = 12V$$

The buck converter reduces the voltage to:

$$V_{out} = 5V$$

Therefore, the voltage reduction is:

$$\Delta V = V_{in} - V_{out} \quad \text{-----} \quad \Delta V = 12 - 5 = 7V$$

The buck converter reduces the input voltage by 7 V.

4.2 Voltage Conversion Ratio

The voltage conversion ratio is:

$$\frac{V_{out}}{V_{in}} = \frac{5}{12} = 0.417$$

Therefore, the output voltage is approximately 41.7% of the input voltage.

4.3 Voltage Reduction Percentage

$$Reduction\% = \frac{V_{in} - V_{out}}{V_{in}} \times 100$$

$$Reduction\% = \frac{12 - 5}{12} \times 100 = 58.3\%$$

Therefore, the buck converter provides approximately 58.3% voltage reduction.

4.4 Motor Supply

The DC motors require: $V_{motor} = 12V$

Since the motor driver is supplied directly from the 12 V power source:

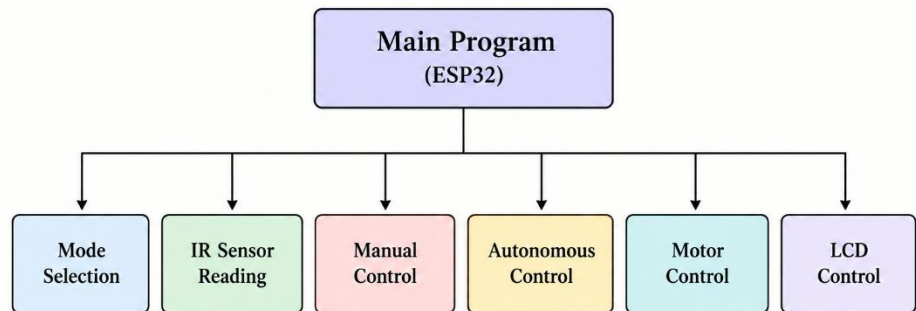
$$V_{driver} = 12V$$

Thus, the motor driver receives the required 12 V supply for the two DC motors.

5. Software Design

5.1 Software Architecture

The software is developed around the ESP32 as the main control unit. The program is organized into several functions responsible for mode selection, sensor reading, manual control, autonomous line following, and motor control. The ESP32 processes the received inputs and generates the appropriate control signals for the robot according to the selected operating mode.



5.2 Main Program Logic

After powering on the robot, the ESP32 initializes the required hardware and software components. The system then determines the selected operating mode. In Manual Mode, the robot responds to commands from the wireless controller. In Autonomous Mode, the ESP32 continuously reads the five IR sensors, processes their readings, and determines the required motor movement. This control process is repeated continuously during operation.

```
11 void application() {
12     if(BT.available() > 0){
13         char command = BT.read();
14         if(command == 'V') {
15             float newKp = BT.parseFloat();
16             float newKi = BT.parseFloat();
17             float newKd = BT.parseFloat();
18
19             Kp = newKp;
20             Ki = newKi;
21             Kd = newKd;
22
23             while(BT.available() > 0 && BT.read() != '\n') {}
24             BT.print(Kp); BT.print(" || "); BT.print(Ki); BT.print(" || "); BT.println(Kd);
25             autono = false;
26         }
27     }
```

```

27     else if(command == 'C') {
28         autono = false;
29         BT.println("Calibrating...");
30         calibrate();
31         BT.println("Done Calibrating");
32     }
33     else if(command == 'A') {
34         autono = true;
35         consts = false;
36     }
37     else if(command == 'Y') {
38         consts = true;
39         autono = false;
40     }
41     else if(command == 'S'){
42         autono = false;
43         consts = false;
44         BT.println("Stopping");
45         stop();
46     }
47 }
48 if(autono) pid();
49 if(consts) basic_auto();
50 }

```

5.3 Manual Mode Implementation

In Manual Mode, the robot is remotely controlled by a team member using an approved wireless controller. The ESP32 receives the control commands and translates them into motor actions such as forward movement, backward movement, left turning, right turning, and stopping. During the official run, the robot is controlled only by the assigned team member without physical interaction with the robot or track.

```

56 void forward(const int& speed) {
57     analogWrite(EN1, speed);
58     analogWrite(EN2, speed);
59     digitalWrite(IN1, 0);
60     digitalWrite(IN2, 1);
61     digitalWrite(IN3, 0);
62     digitalWrite(IN4, 1);
63 }
64
65 void backward(const int& speed) {
66     analogWrite(EN1, speed);
67     analogWrite(EN2, speed);
68     digitalWrite(IN1, 1);
69     digitalWrite(IN2, 0);
70     digitalWrite(IN3, 1);
71     digitalWrite(IN4, 0);
72 }
73

```

```

74 void right(const int& speed) {
75     analogWrite(EN1, speed);
76     analogWrite(EN2, speed);
77     digitalWrite(IN1, 0);
78     digitalWrite(IN2, 1);
79     digitalWrite(IN3, 1);
80     digitalWrite(IN4, 0);
81 }
82
83 void left(const int& speed) {
84     analogWrite(EN1, speed);
85     analogWrite(EN2, speed);
86     digitalWrite(IN1, 1);
87     digitalWrite(IN2, 0);
88     digitalWrite(IN3, 0);
89     digitalWrite(IN4, 1);
90 }

```


5.4 Autonomous Mode Implementation

In Autonomous Mode, the robot operates without human control or intervention. The five IR sensors continuously detect the position of the designated line and provide feedback to the ESP32. The controller processes the sensor readings and adjusts the two DC motors to keep the robot aligned with the track. The process continues throughout the run until the robot reaches the finish point.

```
9  const int baseSpeed = 150;
10 const int target = 2000;
11 const char directions[5][100] = {"Far Right", "Right", "Middle", "Left", "FarLeft"};
12 int sensorValues[5];
13 int normalized_value[5];
14
15 int bits[] = {1, 2, 4, 8, 16};
16
17 int maxValues[] = {0, 0, 0, 0, 0};
18 int minValues[] = {1023, 1023, 1023, 1023, 1023};
19
20 int irThreshold[5];
21
22 int position = 2000;
23
24 int pid_output = 0;
25
26 extern int lastError, I;
27
28 extern float Kp, Ki, Kd;
29
30 extern bool autono;
31 extern bool manual;
32
33 void sensors_init() {
34     pinMode(RR, INPUT);
35     pinMode(R, INPUT);
36     pinMode(M, INPUT);
37     pinMode(L, INPUT);
38     pinMode(LL, INPUT);
39
40     analogReadResolution(10);
41 }
42
43 void read_sensors(int& rr, int& r, int& m, int& l, int& ll) {
44     rr = analogRead(RR);
45     r = analogRead(R);
46     m = analogRead(M);
47     l = analogRead(L);
48     ll = analogRead(LL);
49 }
```

5.5 Control Algorithm

The control algorithm is based on the readings of the five IR sensors. The ESP32 evaluates the sensor states to determine the position of the line relative to the robot. Based on the detected position, the controller generates the appropriate motor control signals to maintain the robot on the designated path. The sensor-reading and motor-control process is continuously repeated to achieve stable and accurate line following.

```
13 float Kp = 2, Ki = 0, Kd = 0;
14
15 const int sensorPins[] = {32, 35, 34, 39, 36}; // left to right
16
17 int P = 0, I = 0, D = 0;
18 extern int irThreshold[5];
19
20 int base_speed = 40;
21
22 void pid() {
23     int active_sensors = 0;
24     long weighted_sum = 0;
25
26     int weights[] = {-2, -1, 0, 1, 2};
27
28     for(int i = 0; i < 5; ++i) {
29         int sensor_val = analogRead(sensorPins[i]);
30         BT.print(sensor_val); BT.print(" || ");
31         int in_line = (sensor_val > irThreshold[i]) ? 1 : 0;
32
33         weighted_sum += in_line * weights[i];
34         active_sensors += in_line;
35     }
36     BT.println();
37
38     int error = 0;
39     if(active_sensors > 0) {
40         error = weighted_sum / active_sensors;
41     }
42     else {
43         error = (lastError > 0) ? 20 : -20;
44     }
45
46     P = error;
47     I += error;
48     D = error - lastError;
49     lastError = error;
50
51     int PID_value = P * Kp + I * Ki + D * Kd;
```